User-defined functions

Awk also allows for the creation of user-defined functions, which can be used within the script. Functions can be declared anywhere within the code, even after they have been used. This is because Awk reads the entire program before executing it.

```
#! /bin/sh
#User defined functions example
awk '
{nodecount()}  # Execute the user-defined nodecount() for each row
to update the node count
$0~/<clusternode /,/<\/clusternode>/{ print } # Print the rows
between "<clusternode " and  "</clusternode>"
function nodecount() #Function definition for nodecount
{if($0~/clusternode name/){count+=1}
}
END{print "Node Count="count}' $1 #Print the count value in the
END pattern which is executed once

sh test.sh cluster.conf
    <clusternode name="node-01.example.com" nodeid="1">
        <fence>
        </fence>
    </clusternode>
    <clusternode name="node-02.example.com" nodeid="2">
        <fence>
        </fence>
    </clusternode>
    <clusternode name="node-03.example.com" nodeid="3">
        <fence>
        </fence>
    </clusternode>
Node Count=3
```

In the example above, we have created a script, test.sh and passed the *cluster.conf* file as the input to it. The script uses the *nodecount()* function before declaring it, and executes it for every row. A counter is updated for each row containing the entry "*clusternode name*". Then it prints the information for each node, using the range-pattern *<clusternode >,</clusternode>*. In the end, the count is displayed in the *END* block. This example also demonstrates how range-patterns in Awk can be used for parsing XML files.

Range-patterns: The range-pattern is a very useful tool for extracting information from text-files. For example, to get the table definitions from a MySQL dump, the range-pattern "*create table* `*"*,";" can be used, since ";" marks the end of a table definition:

```
awk -v IGNORECASE=1 '/CREATE TABLE `/,/;/' dump_file_name
```

Similarly, if we know the text specifying the beginning (*Create Table `<tablename>*) and ending (*Drop table `<next table>*) of a table of data in the MySQL dump file, we can use that as a range pattern to extract the data for that particular table. We can pass the table name as a variable, *tname*, to the script:

```
 awk -v IGNORECASE=1 -v tname=host 'BEGIN{str="^CREATE TABLE
`"}$0~str""tname,/drop table/{ if($0!~/drop table/)print}' dump_
file_name
```

However, for a scenario where some text is contained within tags specified by the same markers (for example, table data between multiple *CREATE TABLE* statements within a dump file), the *next* statement, along with a marker variable, can be used to extract the data. The script below demonstrates this:

```
 #! /bin/sh
awk -v IGNORECASE=1 -v tname=$1 ' #Variable tname provides the
table name
BEGIN{str="^CREATE TABLE `";defstart=0} #Specify the pattern to
match and set the marker variable to default
$0~str""tname{ defstart=!defstart; print;next} #If row matches
the table name, toggle the variable, print the row and switch to
next row without further processing.
defstart==1{if($0~str||$0~/drop table/){exit}else{print}} #If the
marker variable is set, print the rows till the next row with the
create or drop table statement
' $2
```

Let's pass two arguments to this script; the first one is the table name, and the second the dump file. In the *BEGIN* block, we need to specify the pattern (begins with *Create Table* `) and set the marker variable to match the table name to 0. When the pattern and table name are matched in a row, we can toggle the variable, print the row, and use the *next* statement to jump to the next row of the dump file, without processing further statements in the script. Since the marker variable is now set, further statements will be executed for each row, till the pattern is found again in the dump file. Thus, we will get all the data contained within the markers.

In this article, we have seen how the functionality provided by Awk can quickly come to the aid of systems administrators whenever they need to parse, summarise or extract certain data from text files. Awk is fairly easy to learn if you already know C. For those who are interested in learning more about Awk, a very detailed user guide is available at: h*ttp://sunsite.ualberta.ca/Documentation/Gnu/gawk-3.1.0/html_chapter/gawk.html*. END

By: Vishal Bhatia

The author is a Linux and virtualisation consultant, and enjoys working on Linux and open source applications. He's always keen to learn about new technologies, and enjoys watching and playing cricket.